STM32 Embedded Temperature Control

Design and Implementation of an LCD-Based Settings, Sensing, and PWM Demonstrator

Abstract

This project combines two successive firmware stages for an STM32F103RET6 embedded temperature-control demonstrator. The initial stage establishes a button-operated LCD menu, editable temperature settings, flash-backed setpoint storage, and timer-based visual feedback. The second stage extends this interface with UART text reception, I2C temperature acquisition from an MPU6050, and proportional pulse-width modulation (PWM) as an actuator-effort indication. The implementation uses the STM32 hardware abstraction layer and a custom parallel LCD driver. This report explains the system architecture, menu behavior, temperature conversion, and control law, and compares the archived implementation with the supplied design requirements. Analytical examples characterize the configured PWM response. Source inspection identifies incomplete UART parameter handling, a fixed deadband, and build-blocking inconsistencies in the second-stage source. These limitations are retained and documented because the source code has been preserved without modification. A real-hardware video is identified as the intended experimental artifact; it was not present in the supplied files, so no measured performance is inferred.

Keywords: STM32F103; embedded systems; LCD interface; MPU6050; I2C; UART; PWM; flash memory.

Report organization

Section	Focus	Page
1	Objectives and combined requirements	2
2	Hardware and software architecture	3
3	Menu interface and persistent settings	4
4	Temperature measurement and PWM control	5
5	UART interface and timing behavior	6
6	Implementation limitations and build status	7
7	Analytical results and experimental verification	8
8	Repository use, conclusions, and references	9

1. Objectives and Combined Requirements

The objective was to develop the interface and control infrastructure for a simple temperature-control system in two stages. The first stage concentrates on local configuration and user feedback. The second retains that foundation and introduces communication, sensing, and output control. Packaging the stages together presents one development sequence, while keeping the two applications independent and traceable [1-4].

1.1 Initial interface requirements

The first specification calls for a three-option menu on a character LCD, operated by two push buttons. One button moves through choices and the other selects them. The first option opens a Hand Tune / Reserved submenu and permits incremental editing of a desired-temperature value, initially 25. A switch change saves the setting, flashes an indicator three times at 0.5-second intervals using a timer, and returns to the main menu. An inactive submenu must return home after five seconds [1].

The second option similarly edits an offset, initially 2. Negative offsets must be rejected, with an error indicator illuminated for 0.5 seconds when a decrement below zero is attempted. The intended thermal thresholds are centered on the desired temperature: $T_{low} = T_{set}$ - offset and $T_{high} = T_{set}$ + offset. The third option is reserved in this stage. This threshold concept motivates the settings interface; the first-stage program does not yet execute a temperature-control loop [1,3].

1.2 Communication and control extension

The second specification requests manual or UART-based adjustment of both parameters, a framed binary receive protocol, MPU6050 temperature acquisition over I2C, and conversion of the raw signed reading into degrees Celsius. The third option becomes Tune Temp. LED intensity represents actuator effort through PWM proportional to absolute temperature error, with a deadband and a recommended 1 kHz output. Persistent settings combined with PID control are an optional extension; PID is not implemented in the supplied code [2,4].

Requirement group	Implementation represented in the archive
Local settings	Both stages edit a setpoint and offset; only the setpoint is written to flash.
UART parameter updates	Stage 2 implements a text display receiver, rather than the requested framed numeric update protocol.
Temperature / PWM	Stage 2 contains sensor conversion and a proportional output routine with a fixed one-degree threshold.
Experimental evidence	A hardware demonstration is reported by the project owner; no video, readings, or instrument captures were supplied.

Evidence convention: statements about implemented behavior are based on source inspection. Calculated examples are explicitly identified. Neither source presence nor old build artifacts constitute a new hardware validation.

2. Hardware and Software Architecture

Both firmware snapshots target the STM32F103RET6 (LQFP64). The linker scripts describe 512 KiB of flash and 64 KiB of RAM. Application clock configuration selects the internal 8 MHz HSI oscillator without a PLL and uses undivided peripheral buses. HAL initialization, GPIO setup, timer setup, LCD initialization, and setpoint restoration precede the foreground menu loop [3,4].

Subsystem	Pin assignment or configuration
Character LCD	RS PC0; R/W PC1; E PC2; D4-D7 PC3-PC6. Four-bit parallel transfer.
User input	PB11: cycle/increase; PB12: enter/decrease/back; PB13/PB14: save-trigger switches.
Status indicators	PC8: save indication; PC9: offset-underflow indication; PB15: nonzero stage-2 effort.
Temperature sensor	I2C1 on PB6 (SCL), PB7 (SDA), 100 kHz; MPU6050 7-bit address 0x68.
Serial interface	USART1 on PA9 (TX), PA10 (RX); 9600 baud, 8N1, no flow control.
PWM output	PA6, TIM3 channel 1 in stage 2; channel 4 on PB1 is configured but not used by the control routine.

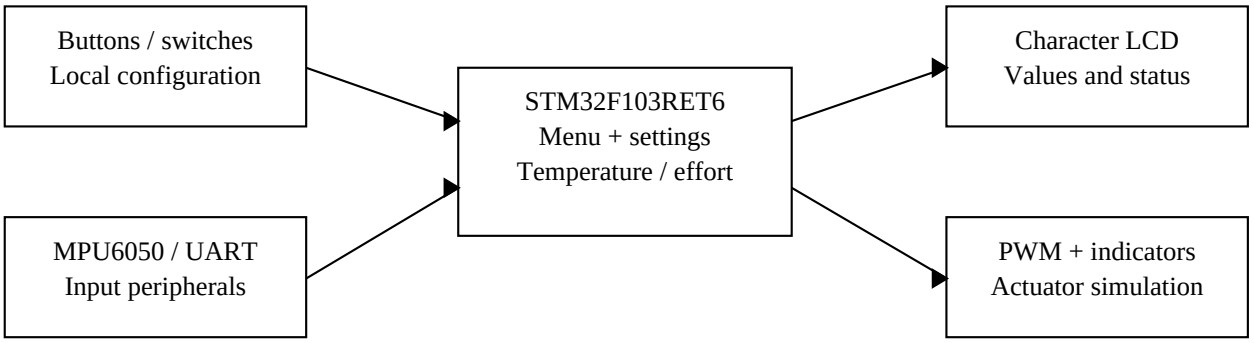

Figure 1. Functional architecture derived from the firmware; this is not a physical wiring schematic.

The custom LCD driver splits commands and characters into two nibbles and writes them through GPIO. Its positioning routine supports several row addresses, while the application uses two rows and 17-byte line buffers for a 16-character display. Short HAL delays are embedded in the LCD driver, so display updates add blocking time even when other activities use interrupts [3,4].

The stage-2 source contains conflicting duplicated GPIO routines. The table records the intended application mapping, including assignments from the later routine. Stage 1 enables input pull-ups; stage 2 uses GPIO_NOPULL and depends on stable external input bias. The exact development-board model and physical LED wiring cannot be established from the supplied files.

3. Menu Interface and Persistent Settings

3.1 State-based user interaction

The initial firmware represents the interface with an enumerated menu state, separate main-menu and submenu indices, and a display-dirty flag. PB11 cycles among the three main entries. PB12 enters the selected branch. Inside the numerical editing screens, PB11 increments and PB12 decrements. Both edited values are bounded between 0 and 999. A falling-edge comparison detects a new button press; stage 1 also applies a 200 ms input lockout [3].

Only two entries are visible at once. When the third main item is selected, the window shifts to show the second and third entries, and a leading greater-than symbol identifies the selection. Stage 1 retains Option3 as a placeholder. In stage 2, its displayed name is Tune Temp and it opens the live sensor/output screen. The first branch remains Hand Tune / Reserved; the second becomes Offset / UART [3,4].

Event	Stage-1 behavior	Stage-2 behavior
PB11 / PB12 in editing	Increment / decrement; 200 ms lockout.	Increment / decrement; no equivalent explicit lockout.
Inactivity	Five-second return outside main menu.	Five-second return outside main and UART, including Tune Temp.
Offset at zero + decrement	Reject; PC9 pulse for 500 timer ticks.	Reject; blocking PC9 pulse for 500 ms.
PB13 or PB14 change	Save if changed; timed blink; return home.	Save if changed; blocking blink; return home.

3.2 Flash storage strategy

A helper reads the device flash-size register at 0x1FFFF7E0 and locates the last flash page. Saving unlocks flash, erases that page, and programs two halfwords: the marker 0xA5A5 followed by the desired-temperature value. On startup, a missing marker selects the default value 25; a matching marker restores the stored halfword. The last-saved value is cached so repeated save events without an edit avoid another erase [3,4].

Only hand_tune_value is persisted. The offset is initialized to 2 and is not stored, despite the broader parameter-storage goal. Saves can also occur on an inactivity return. Flash erase and program return codes are discarded, so the success indication reflects completion of the attempted sequence rather than verified storage. The existing marker is not a checksum and the loaded value is not range-checked [3,4].

The linker scripts expose the full flash capacity without reserving the last page used by the application. The supplied small firmware may leave that page unused, but the memory layout does not formally prevent future program growth into the settings region. A future revision should reserve the page and validate both writes and restored values; this packaging retains the original strategy.

4. Temperature Measurement and PWM Control

4.1 MPU6050 access and temperature conversion

On first entry to Tune Temp, the firmware reads WHO_AM_I at register 0x75 and compares it with 0x68. A matching response causes a zero write to power-management register 0x6B. The HAL transaction address is 0xD0, the left-shifted form of 7-bit address 0x68. Two bytes from 0x41 and 0x42 are combined high-byte first and interpreted as a signed 16-bit raw temperature [4].

$$T_{actual}\ [degrees\ C] = T_{raw} / 340 + 36.53$$

The code continuously displays actual temperature and the desired value while this screen is active, adding a 100 ms delay after each update. I2C transfers and LCD operations increase the full loop duration beyond 100 ms. The sensor reading is the device's internal temperature; the archive does not establish calibration against ambient temperature. Read and write return values are ignored, and the initialization flag is set even if the identity check or bus transactions fail [4].

4.2 Proportional output law

Let $e = |T_{actual} - T_{set}|$. The PWM compare value u is 0 for e < 1 degree C; otherwise u = min(999, floor(50e)) for ordinary finite readings. This is an absolute-error effort signal: it contains no heating/cooling direction and no integral or derivative term. The editable offset is not read by this routine; DEADBAND is fixed at 1.0. There is no stateful hysteresis of the kind illustrated in the first specification [1,4].

With timer clock 8 MHz, prescaler PSC = 7, and auto-reload ARR = 999, the nominal PWM frequency is 8,000,000 / [(7 + 1)(999 + 1)] = 1,000 Hz. Duty ratio is u / 1000. The compare clamp therefore reaches 99.9%, rather than an exact 100%. PB15 is set high whenever u is nonzero; variable-duty output is on PA6, so those pins serve different purposes [4].

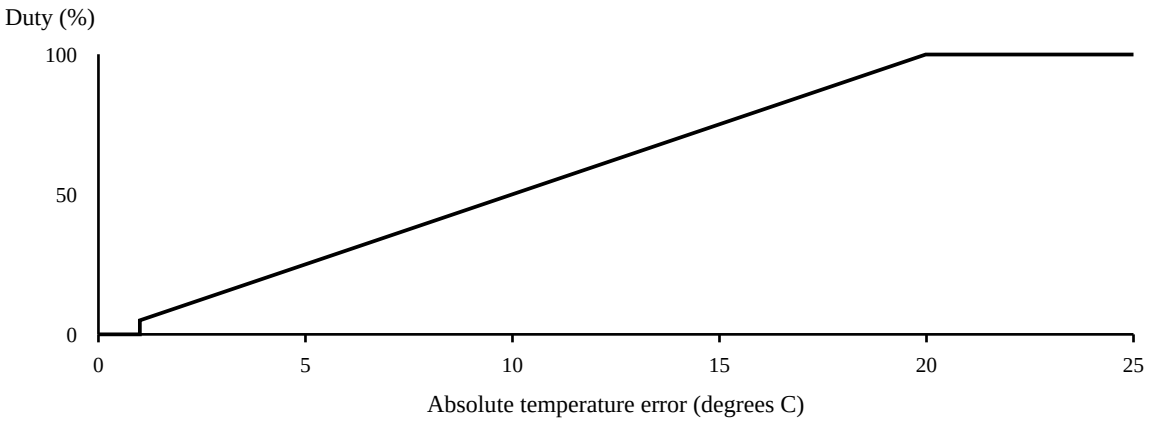

Figure 2. Analytical envelope of the configured output law; integer compare steps are not resolved. This is not a measured response.

5. UART Interface and Timing Behavior

5.1 Requested binary parameter protocol

The extension specification calls for UART entry in both desired-temperature and offset settings. The receiver must wait up to ten seconds for start byte 0xEE, accept a two-byte uint16_t value and apply it, and continue until end byte 0xAA. A byte arriving before the expected start marker must produce Wrong Packet on the LCD and return the parser to its waiting state. The specification does not explicitly establish the byte order of the two-byte value, so an interoperable implementation would need to define that detail [2].

5.2 Receiver present in the supplied source

The stage-2 UART branch is exposed under Option2 only. It receives one byte at a time with a HAL_UART_Receive timeout of 10 ms. Bytes are collected in a 17-byte buffer, limited to 16 display characters. A carriage return, newline, or full buffer marks the message as received, and the LCD shows Received: followed by the text. Until a complete message is marked received, ten seconds from screen entry leads back to the main menu [4].

There is no parser for 0xEE, no extraction of a two-byte numeric value, no handling of 0xAA, and no Wrong Packet message. Received data are not assigned to either hand_tune_value or offset_value. Once a text message has been marked received, the ten-second condition no longer applies and the user leaves with PB12. This is a serial text-display demonstration, with a different function from the requested parameter protocol [4].

5.3 Timing changes between development stages

In stage 1, TIM3 uses PSC = 799 and ARR = 9 at 8 MHz, giving one interrupt every millisecond. The interrupt callback counts 500 ticks per LED transition and completes six half-periods for three save flashes. A separate counter ends the offset-error pulse after 500 ms. The foreground menu is disabled during the save blink while the interrupt performs its timing [3].

The second stage reuses TIM3 for the 1 kHz PWM time base and replaces the status timing with blocking HAL_Delay calls. Its save indication uses three cycles of 200 ms on and 200 ms off, giving approximately 1.2 seconds overall. The error pulse blocks for 500 ms. TIM1 is configured but no base-interrupt start call drives these jobs. Consequently, the second-stage source does not retain the first specification's timer-driven 0.5-second save indication [1,4].

Tune Temp is also subject to the five-second inactivity return, despite its continuous display refresh. PB12 exit and inactivity timeout explicitly set compare to zero, stop channel 1, and clear PB15. The separate switch-save path returns home through the blink helper without that cleanup, which can leave the last output active after the control screen is gone [4].

6. Implementation Limitations and Build Status

The preserved source is a record of the submitted implementation. Several issues affect reproducibility and behavior; describing them prevents the combined repository from implying full compliance or a newly validated firmware release.

Finding	Evidence and consequence
Missing menu declarations	Stage-2 temperature_control.c uses MenuState and MENU_* constants without any declaration in the supplied application or headers. The first stage has its own enum, but these are separate applications.
Duplicate initialization functions	Stage 2 defines MX_I2C1_Init, MX_TIM1_Init, MX_TIM3_Init, MX_USART1_UART_Init, and MX_GPIO_Init twice. Each pair is in the same translation unit, creating C redefinition errors.
Conflicting GPIO setup	The earlier stage-2 GPIO routine omits PB13/PB14 and PC9; the later routine includes them. Both use no input pull resistors. There is no unique executable configuration until duplicates are resolved.
Display buffer size	The stage-2 Set Point format with a three-digit value produces 17 characters plus a terminator, exceeding display_buf[17]. The default two-digit value fits.
Unchecked peripheral operations	I2C status is ignored before using raw_data, and the sensor flag can be set after a failed wakeup. Flash errors are also ignored; neither path verifies successful operation.
Incomplete parameter behavior	UART does not apply numeric updates; offset is neither persisted nor used by the PWM deadband.
Output lifecycle	Save-switch return can bypass PWM shutdown. Tune Temp also times out after five seconds of button inactivity.

6.1 Reproducibility constraints

Both IDE project descriptions retain the internal name phase_one. Separate STM32CubeIDE workspaces avoid an import-name collision while preserving metadata. The supplied .ioc files are configuration snapshots, with settings that differ from the handwritten application. For example, stage-1 timer parameters in the application differ from its .ioc, and the later stage-2 GPIO additions are not fully represented in the generated section. Regeneration should therefore not be treated as a faithful reconstruction of the archived application [3,4].

Both MSP initialization files disable JTAG and SWD through the original remapping call. Existing Debug binaries and listings were present in the input, but they cannot prove that the currently supplied source compiles or corresponds exactly to the demonstrated firmware. No fresh cross-compilation or physical-board test was performed in preparing this package. The missing declarations and duplicate definitions are direct static findings, not a compiler log. All original code bytes remain unchanged.

7. Analytical Results and Experimental Verification

7.1 Calculated behavior

The following values follow directly from the conversion and PWM equations. They provide reference cases for later bench checks, not measurements. For raw sensor values -340, 0, and 340, the conversion yields 35.53, 36.53, and 37.53 degrees C, respectively. A raw value of zero therefore represents 36.53 degrees C under the implemented formula; it must not be interpreted as zero degrees C [4].

Absolute error (degrees C)	Compare u	Nominal duty (%)
0.00	0	0.0
0.50	0	0.0
1.00	50	5.0
2.00	100	10.0
5.00	250	25.0
10.00	500	50.0
20.00	999	99.9

The strict less-than comparison turns output on at the one-degree boundary, producing a jump from zero to approximately 5% duty. Outside that threshold, duty grows by about five percentage points per degree until the compare clamp is reached near 19.98 degrees of error. Because error magnitude is used, equal deviations above and below the setpoint generate the same command. This characterizes an LED actuator-effort demonstration; it does not demonstrate convergence of a heated or cooled physical process.

7.2 Hardware evidence and verification plan

The project owner reports a real-world implementation and intends to include its demonstration video. No recording was present in the supplied folder. Results/ is therefore reserved for that recording. There are no supplied time-series measurements, calibration records, or oscilloscope captures from which to estimate temperature accuracy, settling time, overshoot, or measured PWM frequency.

Bench check	Evidence to record; status during packaging
Interface / storage	Show navigation, setpoint edits, switch-save flashes, reset/restart, and restored setpoint. Not rerun.
Offset protection	Attempt decrement at zero; record PC9 duration and offset after restart. Not rerun.
Temperature / PWM	Record displayed actual/set temperatures and PA6 waveform for several error magnitudes. Not measured.
Communication	Show current text reception separately from any future binary-protocol repair. Not rerun.
Exit behavior	Test PB12, five-second timeout, and save-switch transitions while PWM is active. Not rerun.

8. Repository Use and Conclusions

8.1 Combined repository organization

The repository contains one report and two firmware snapshots under Code/. The application files have descriptive names: menu_and_settings.c in 01_Menu_and_Settings/Core/Src/ and temperature_control.c in 02_Temperature_Control/Core/Src/. Their supporting headers, LCD driver, interrupt handlers, startup assembly, linker scripts, CMSIS files, HAL drivers, and CubeIDE configuration are retained. The two applications each define main() and must not be linked as one executable.

All 178 retained source, configuration, and license files were copied with identical bytes. Code/Source_Manifest.csv maps every original path to its packaged path and records its SHA-256 value. Generated Debug outputs are omitted. Standard filenames such as main.h and lcd.h remain unchanged because the preserved source includes them by name. README.md provides a GitHub-rendered overview alongside the requested Readme.tex. Results/ contains a tracking placeholder until the actual hardware video is added.

8.2 Reproduction and interpretation

The retained configuration records STM32CubeMX 6.15.0 and STM32Cube FW_F1 V1.8.6; original generated build files identify GNU Tools for STM32 13.3.rel1. Import each stage into a separate STM32CubeIDE workspace and confirm that the renamed application file is included. Stage 2 requires a separately maintained repair before a successful new build can be expected. Any future repair should resolve the menu declarations and duplicated initialization code first, then address UART framing, offset/deadband integration, checked peripheral access, display bounds, and consistent output shutdown.

The combined work demonstrates the progression from a local embedded user interface to a sensor-informed output calculation. Its main contributions are menu-state management, persistent setpoint handling, raw-to-engineering-unit conversion, and timer-based PWM generation. The source also shows the practical importance of preserving timing requirements and integrating communication and output-state handling consistently. The available evidence supports these architectural and code-level conclusions; it does not establish complete requirement compliance or measured thermal-control performance.

References and implementation evidence

[1] Supplied first-stage design specification, Persian-language PDF, four pages; technical requirements on pages 3-4. Institutional cover and administrative material are omitted from this report.

[2] Supplied second-stage design specification, Persian-language PDF, four pages; communication, sensing, PWM, and optional-extension requirements on pages 2-4.

[3] Preserved first-stage firmware: Code/01_Menu_and_Settings/. Principal evidence: Core/Src/menu_and_settings.c, Core/Src/lcd.c, Core/Inc/lcd.h, interrupt/MSP files, linker script, and project configuration.

[4] Preserved second-stage firmware: Code/02_Temperature_Control/. Principal evidence: Core/Src/temperature_control.c, Core/Inc/main.h, Core/Src/stm32f1xx_hal_msp.c, LCD driver, linker script, and project configuration. Equations and nominal values in this report are derived from this source.

Third-party STM32 HAL and CMSIS notices and license files remain with the drivers. The report assigns no new license to the application code.